

Course

« *Computer Vision* »

Sid-Ahmed Berrani

2025-2026



Deep Learning for Computer Vision

Image perception by humans

⇒ The Gestalt theory :

We do not see pixels; we see objects.

We perceive object in their entirety before their individual parts

Gestalt theory says that human perception does not process visual scenes as isolated pixels or points. Instead, we naturally organize visual information into **meaningful structures**: objects, groups, contours, foreground, and background.

⇒ Directly related to **image segmentation**:

Goal: Divide an image into meaningful regions.

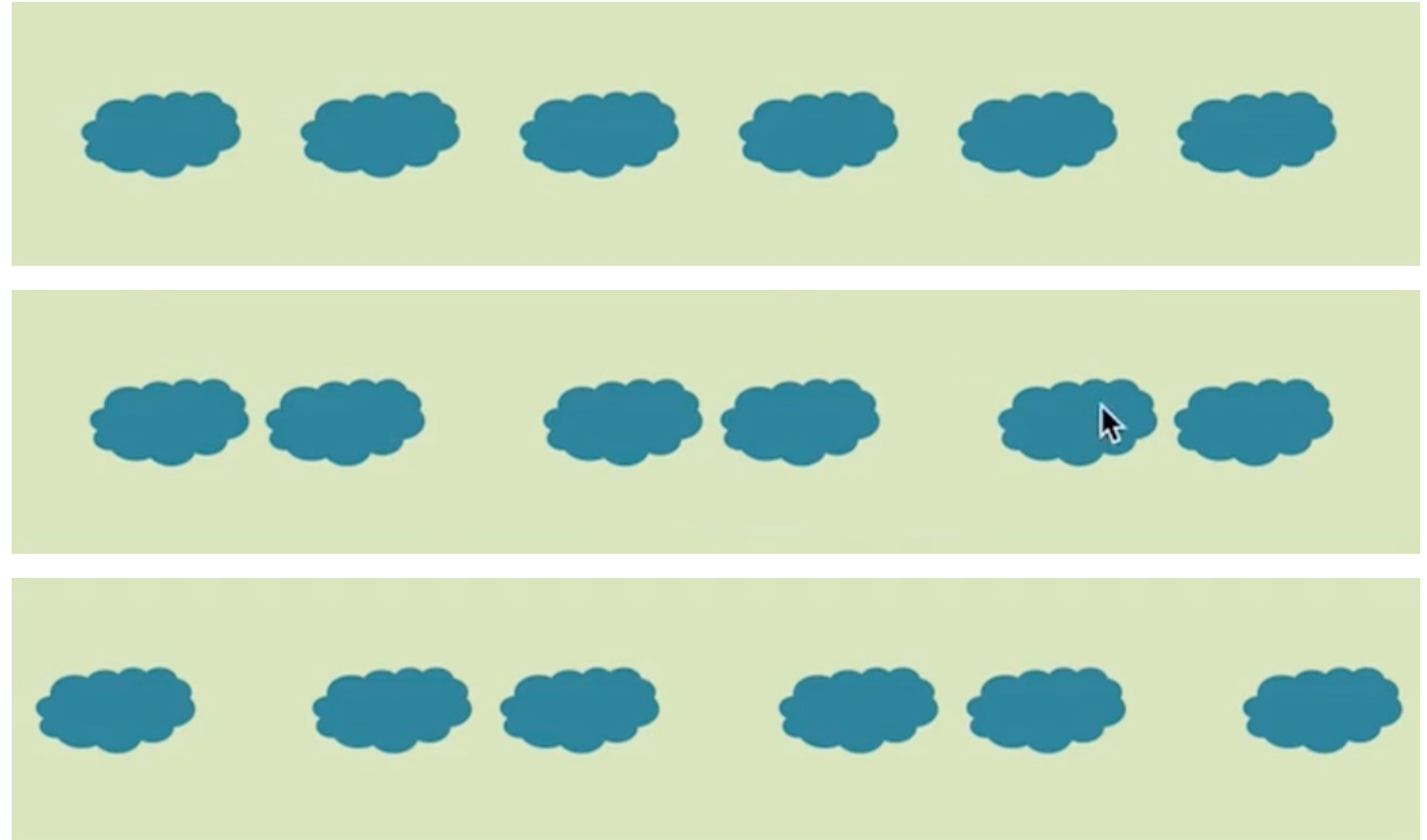


Deep Learning for Computer Vision

Image perception

- **Proximity principle:**

Closer objects are grouped together

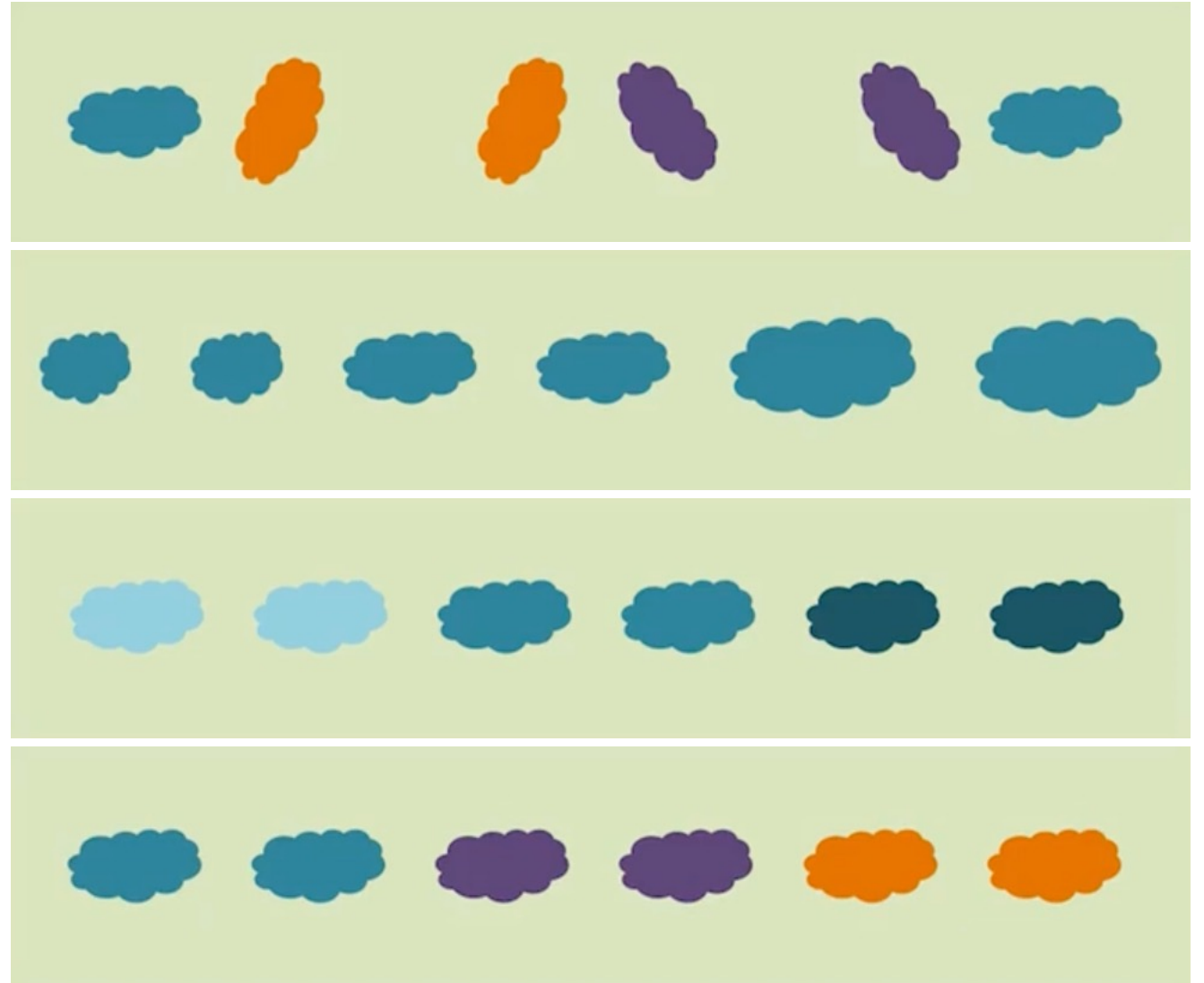


Deep Learning for Computer Vision

Image perception

- **Similarity principle:**

Similar objects are grouped together

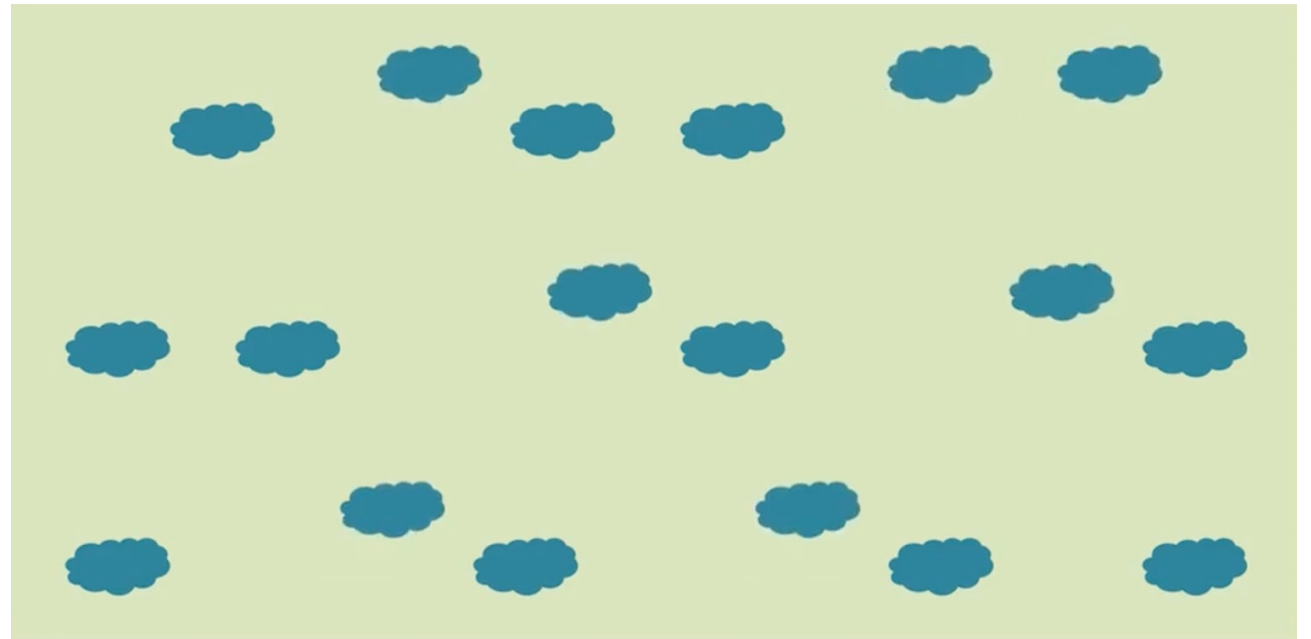


Deep Learning for Computer Vision

Image perception

- **Common fate:**

Objects with similar motion or change in appearance are grouped together

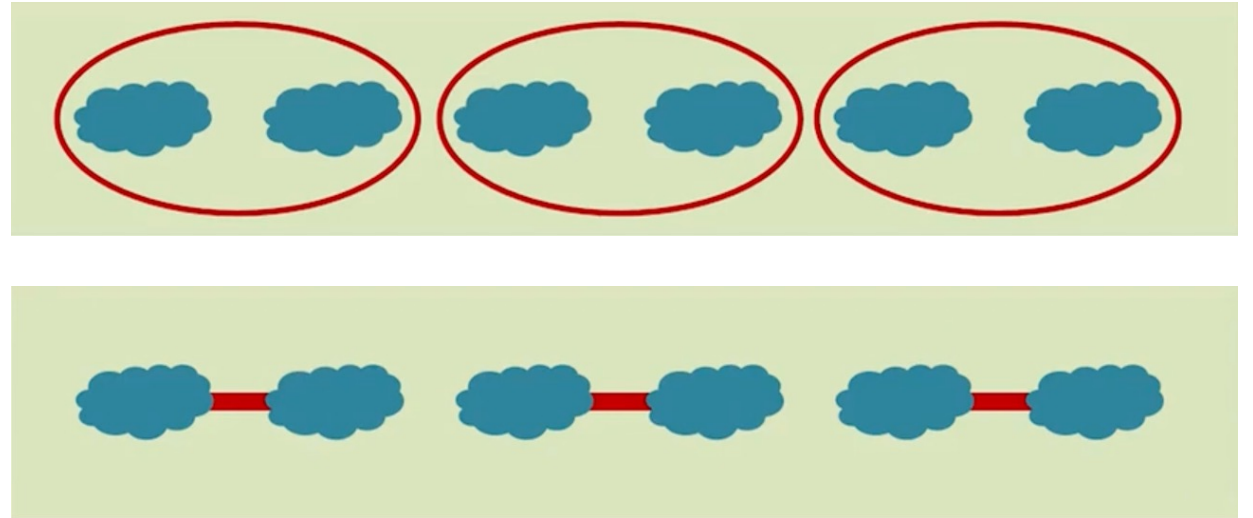


Deep Learning for Computer Vision

Image perception

- **Common region/connectivity:**

Connected objects are grouped together

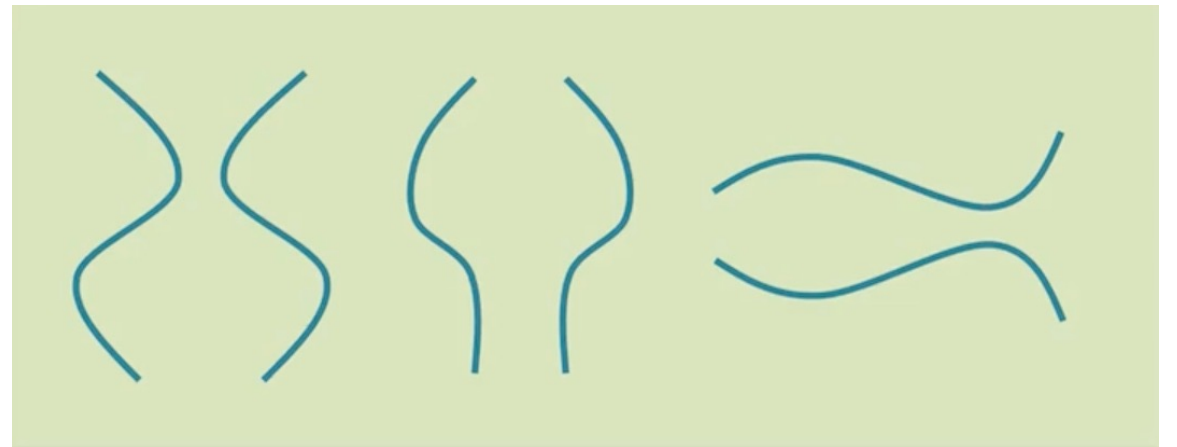


Deep Learning for Computer Vision

Image perception

- **Symmetry principle:**

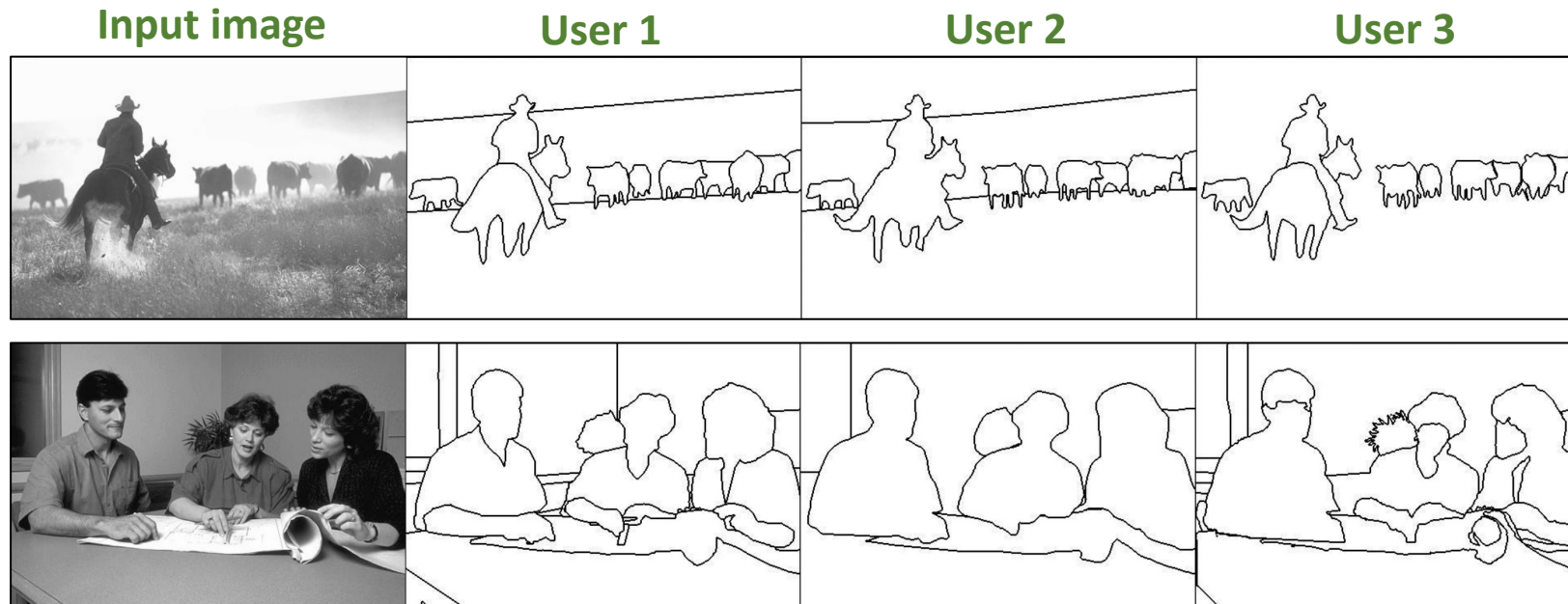
*Parallel and symmetrical features
are grouped together*



Deep Learning for Computer Vision

Image segmentation

- A highly subjective process



Deep Learning for Computer Vision

Image segmentation: Two main strategies

Classical CV

1. **Bottom-up segmentation:** Pixels belong together because they look similar.

color similarity, texture similarity, local continuity,
edge detection, region growing, clustering.

Deep learning

2. **Top-down segmentation:** Pixels belong together because they come from same object.

Object-level reasoning.

The segmentation is guided by: semantics, prior knowledge, learned object representations.

Deep Learning for Computer Vision

Image segmentation

Basically :

Image segmentation => A clustering problem

Each pixel <- a set of visual feature (brightness, color...), but not only...

We can add:

Position

Texture

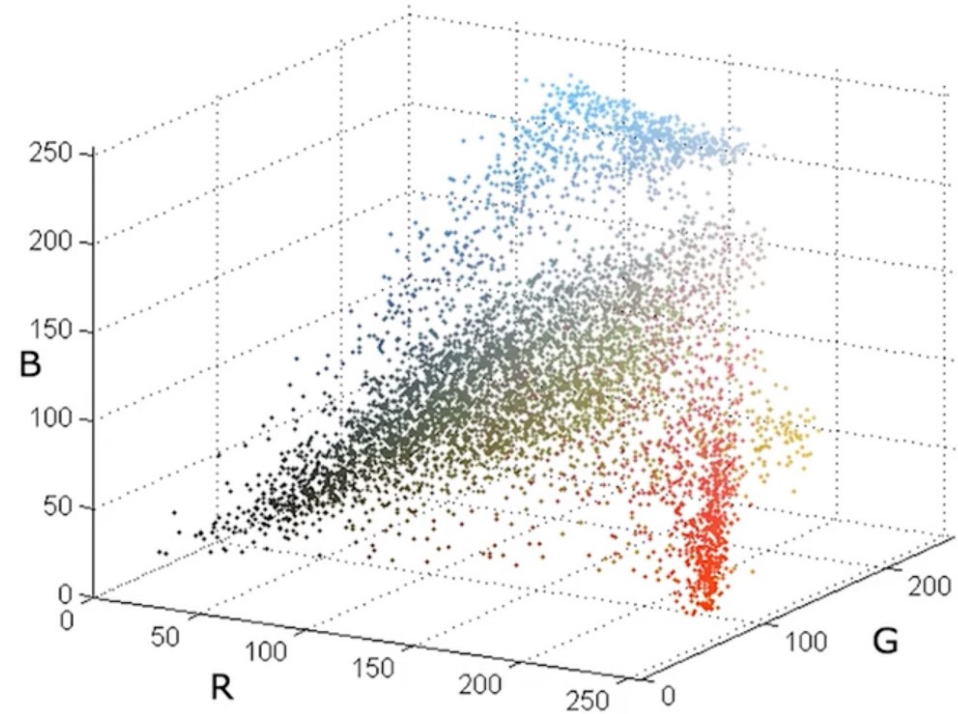
Depth

Motion info

...

Deep Learning for Computer Vision

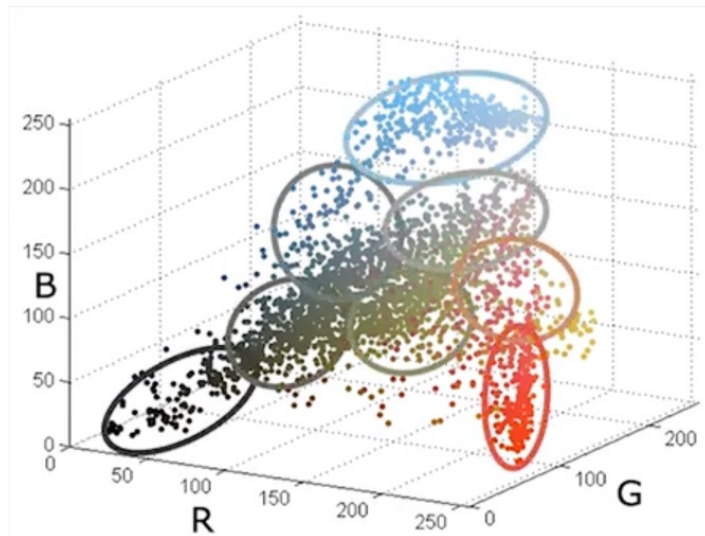
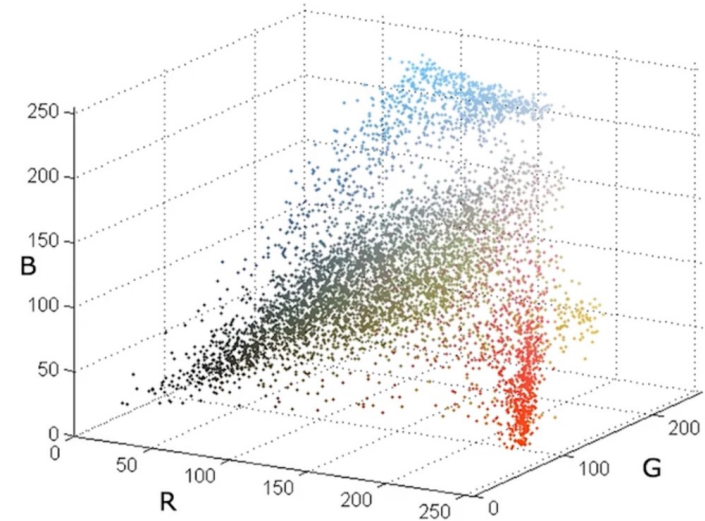
Image segmentation



- ⇒ Pixels as feature vectors **[R, G, B, x, y, d, m, t...]**
- ⇒ Pixel similarity = distance between feature vectors.

Deep Learning for Computer Vision

Image segmentation



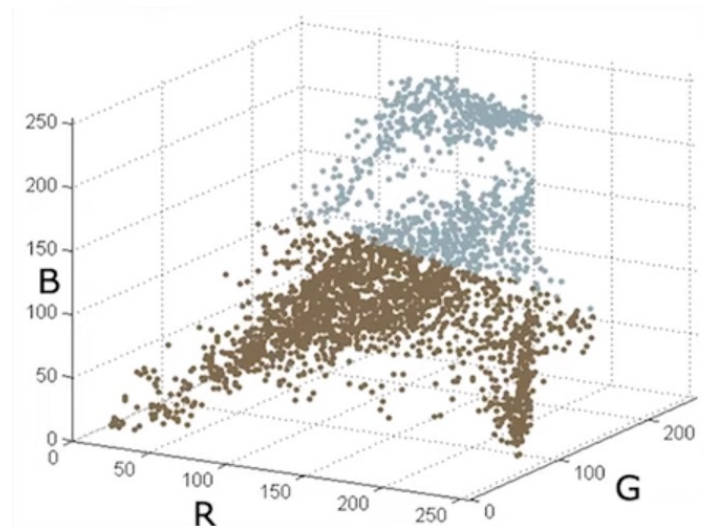
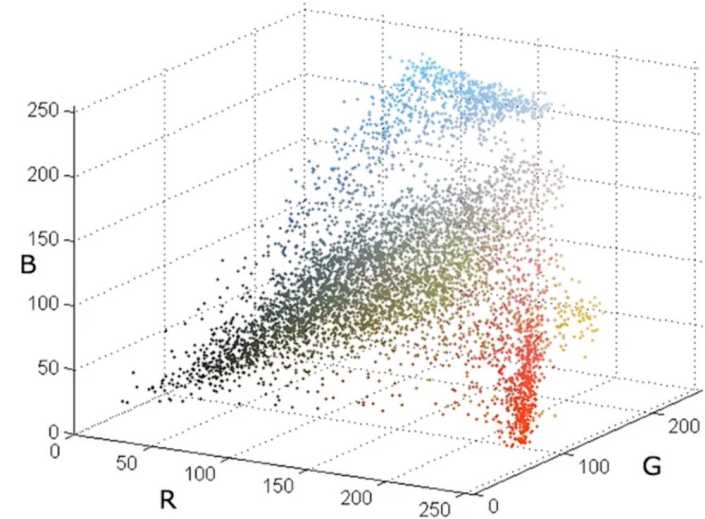
Deep Learning for Computer Vision

1. Image segmentation using K-Means

$K = 2$

Feature vector = $[R, G, B]$

Distance : L2



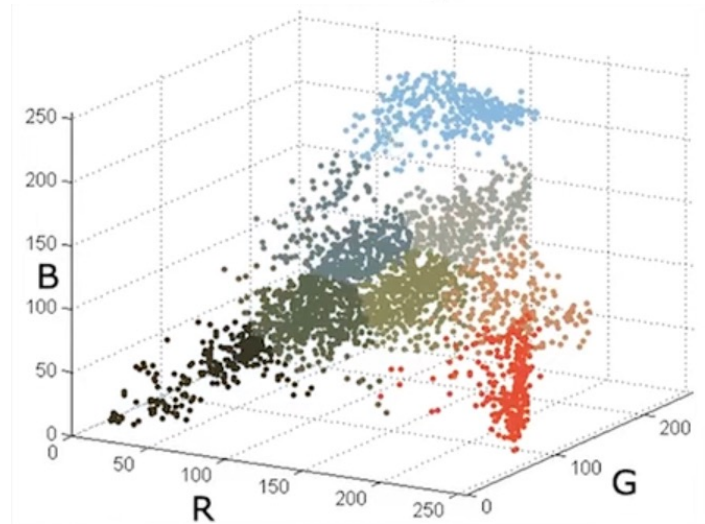
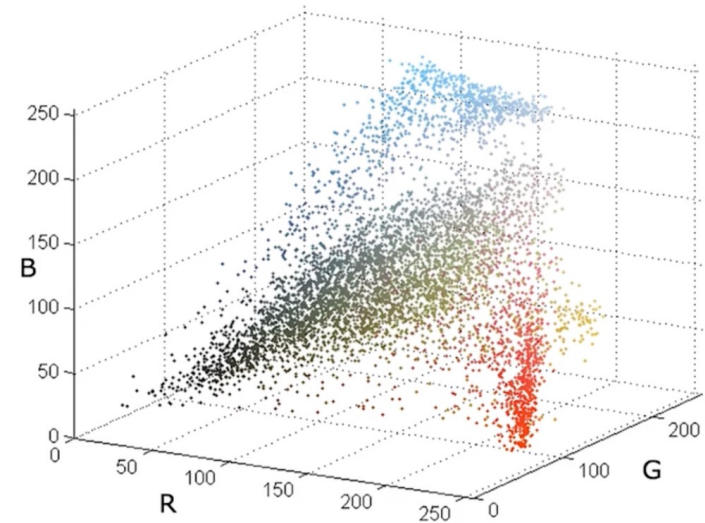
Deep Learning for Computer Vision

1. Image segmentation using K-Means

$K = 8$

Feature vector = $[R, G, B]$

Distance : L2



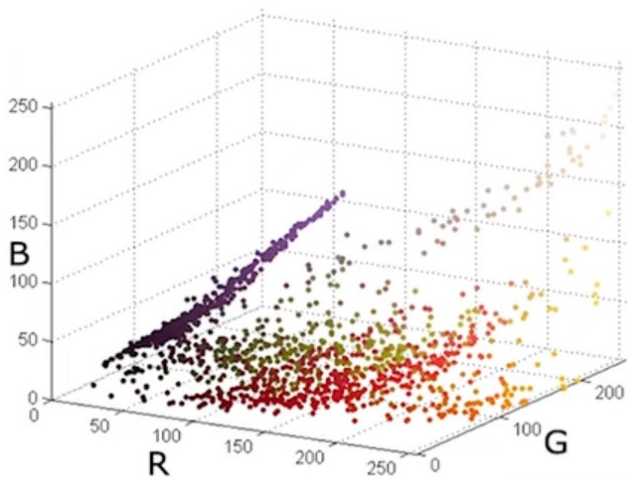
Deep Learning for Computer Vision

Image segmentation using K-Means

$K = 16$



Feature vector = $[R, G, B]$



Feature vector = $[R, G, B, x, y]$

Deep Learning for Computer Vision

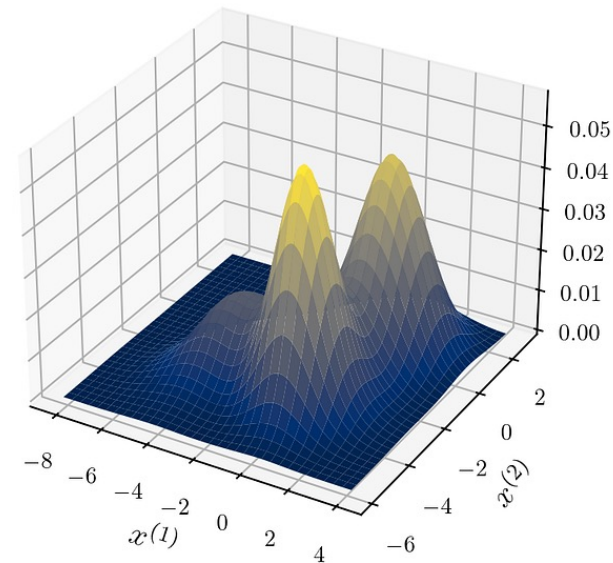
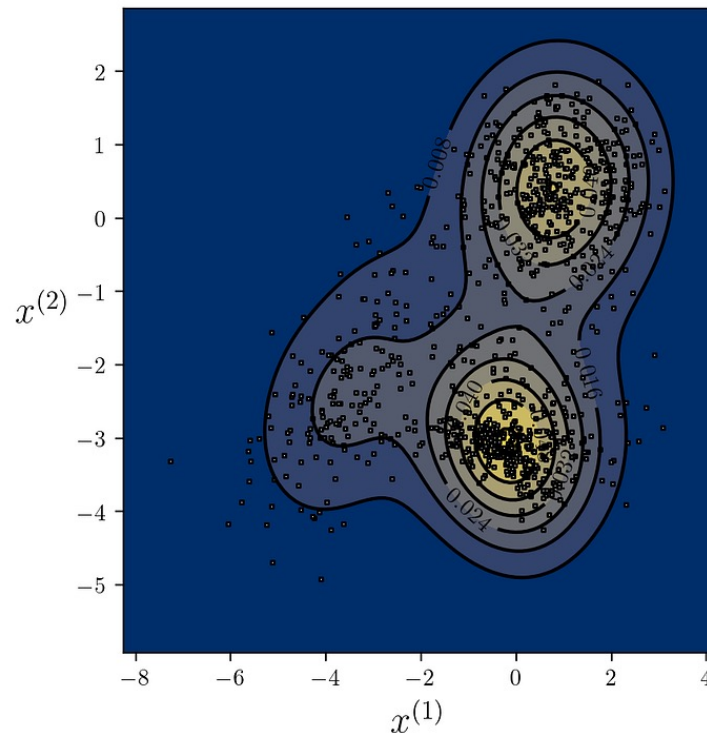
Image segmentation using K-Means

- Simple & reasonably fast
- Need to choose the value of K
- Sensitive to initialization
- Sensitive to outliers.

Deep Learning for Computer Vision

Image segmentation using Mean Shift

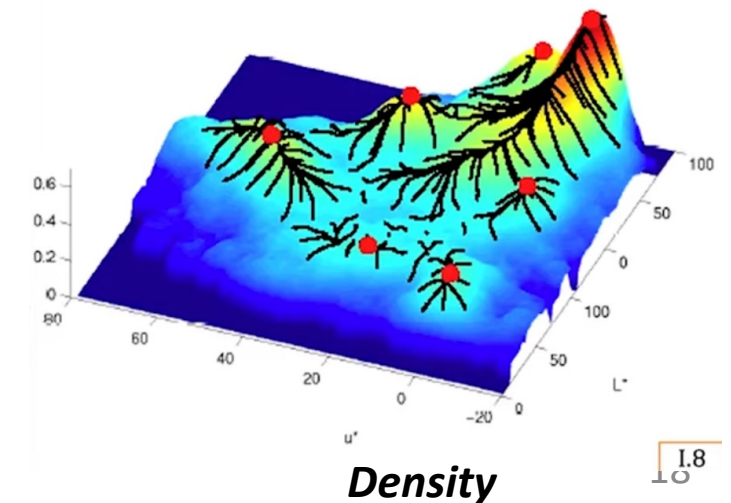
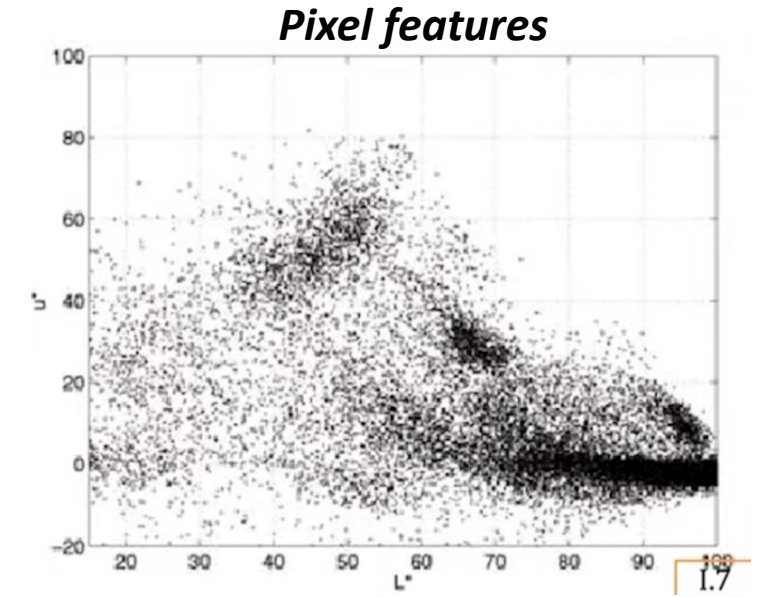
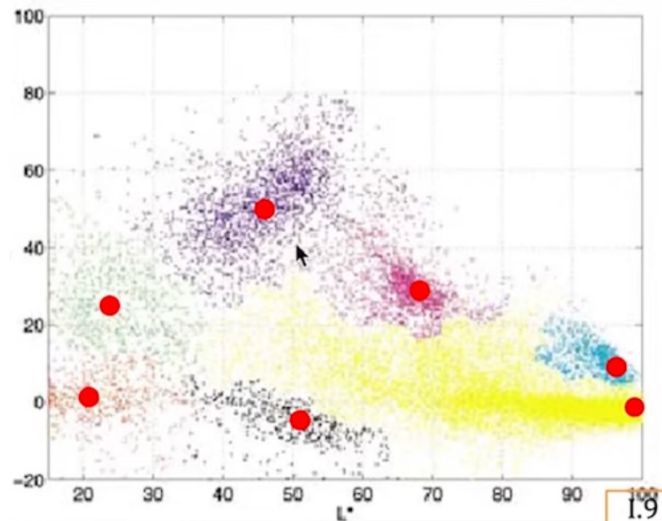
- Relies on kernel density estimation:



Deep Learning for Computer Vision

Image segmentation using Mean Shift

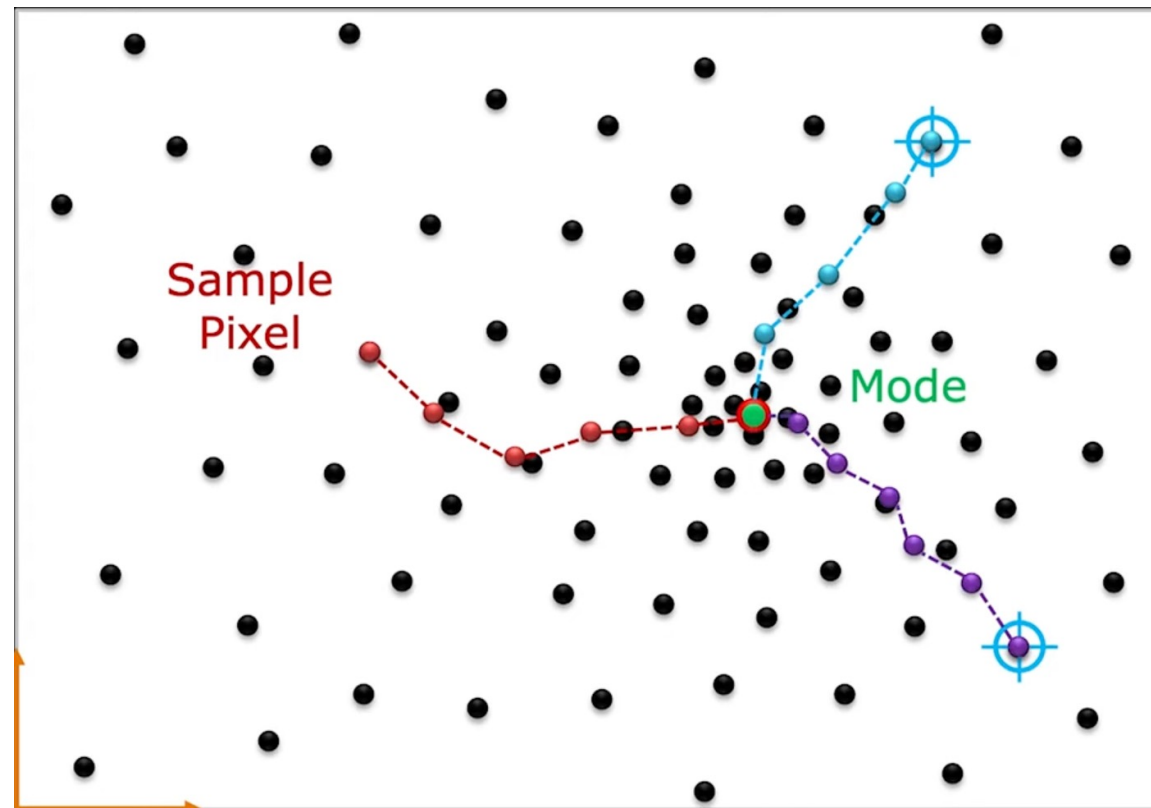
- Each hill represent a cluster (a segmented region).
- Peaks of the hill are cluster representatives.
- To assign pixels, each pixel climbs the neighboring hill.



Deep Learning for Computer Vision

Image segmentation using Mean Shift

- How to determine to which cluster (image region) a pixel belongs to?



Deep Learning for Computer Vision

Image segmentation using Mean Shift

Given: Distribution of N pixels in feature space (e.g. $\mathbf{f} = (\mathbf{x}, \mathbf{y}, \mathbf{R}, \mathbf{G}, \mathbf{B})$).

Task: Find density modes (clusters) in the feature space.

1: Initialize each mean using the pixel feature vector: $\mathbf{m}_i = \mathbf{f}_i$

2: Repeat the following steps for each mean $\mathbf{m}_i$:

a: Place a kernel window of size $\mathbf{W}$ around $\mathbf{m}_i$

b: Compute the local centroid $\mathbf{m}$ inside the window and assign $\mathbf{m}$ to $\mathbf{m}_i$ $\mathbf{m}_i = \mathbf{m}$

c: Stop if the shift in $\mathbf{m}_i$ is smaller than ϵ : $|| \mathbf{m}_i^{(t+1)} - \mathbf{m}_i^{(t)} || < \epsilon$

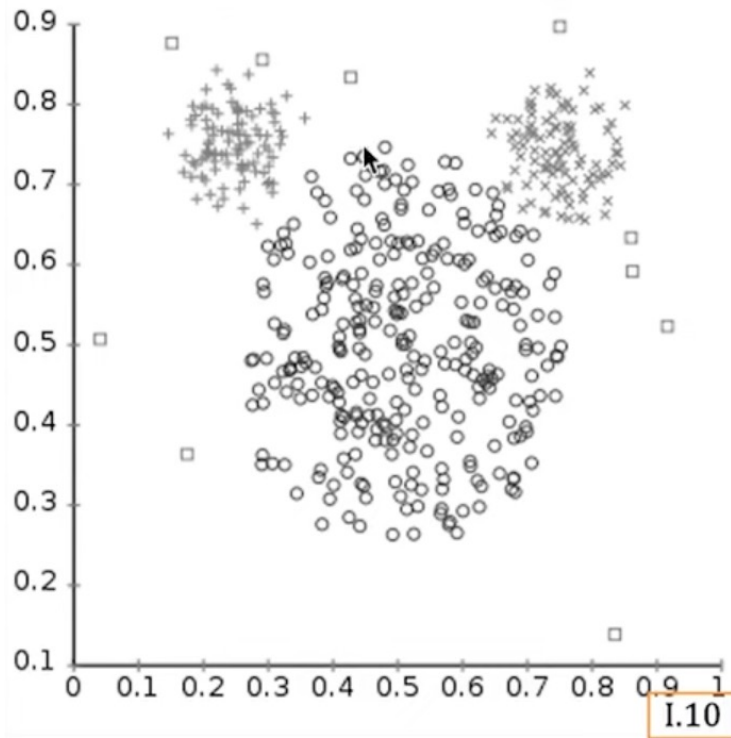
3: Points converging to the same mode belong to the same cluster (segment).

Key Idea:

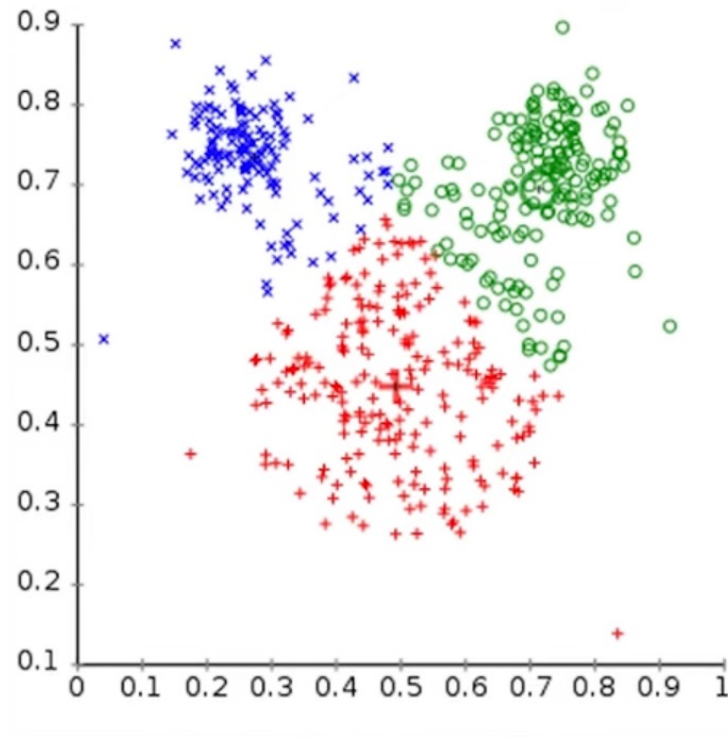
Points iteratively move toward regions of high local density in feature space.

Deep Learning for Computer Vision

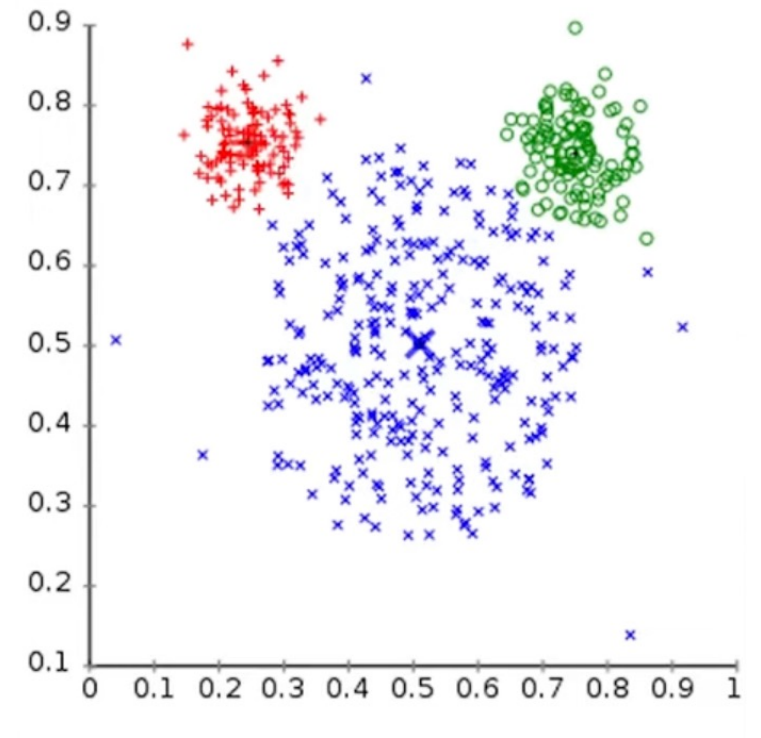
Image segmentation using Mean Shift



Original data



K-mean with $k=3$



Mean shift

Deep Learning for Computer Vision

Image segmentation using Mean Shift



Input image



*K-mean segmentation
($k = 16$)*



*Mean shift segmentation
($W = 21$)*

Deep Learning for Computer Vision

Image segmentation using Mean Shift



Deep Learning for Computer Vision

Image segmentation using Mean Shift

- Simple but computationally expensive
- Finds arbitrary numbers of segments (clusters).
- No initialization required.
- Robust to outliers.
- Results depend on **W** .

Deep Learning for Computer Vision

Image segmentation: Many other classical methods

Main Limitation

❑ Hand-crafted rules often fail on:

- Complex scenes
- Illumination changes
- Texture variations
- Object variability

Deep Learning for Computer Vision

Image segmentation: From Classical Vision to Deep Learning

❑ Classical methods rely on:

- Manually designed features
- Local similarity rules
- Fixed heuristics

❑ Deep Learning methods learn:

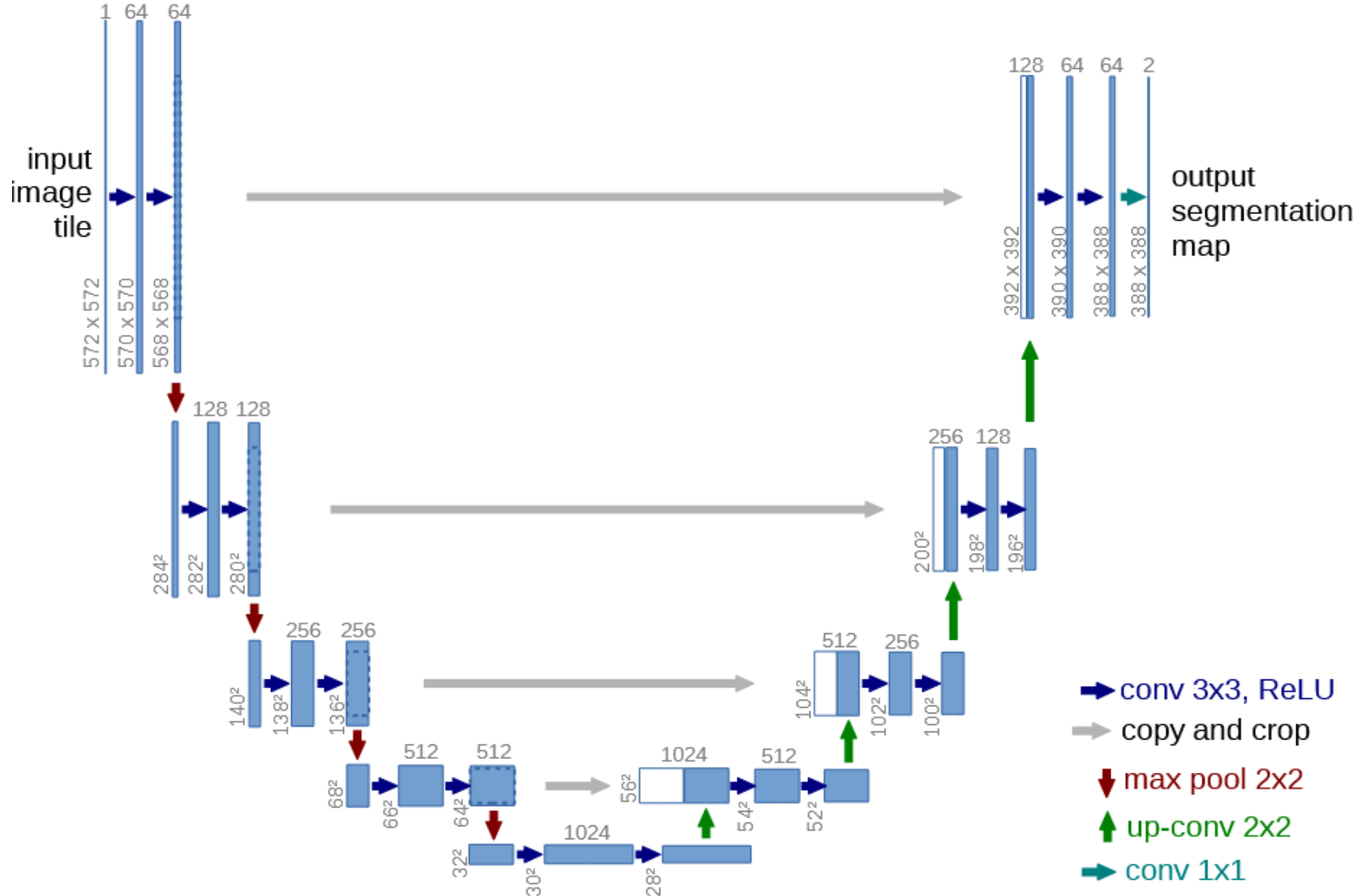
- Object representations
- Semantic understanding
- Hierarchical visual features

Instead of defining segmentation rules manually, the model learns them directly from data.

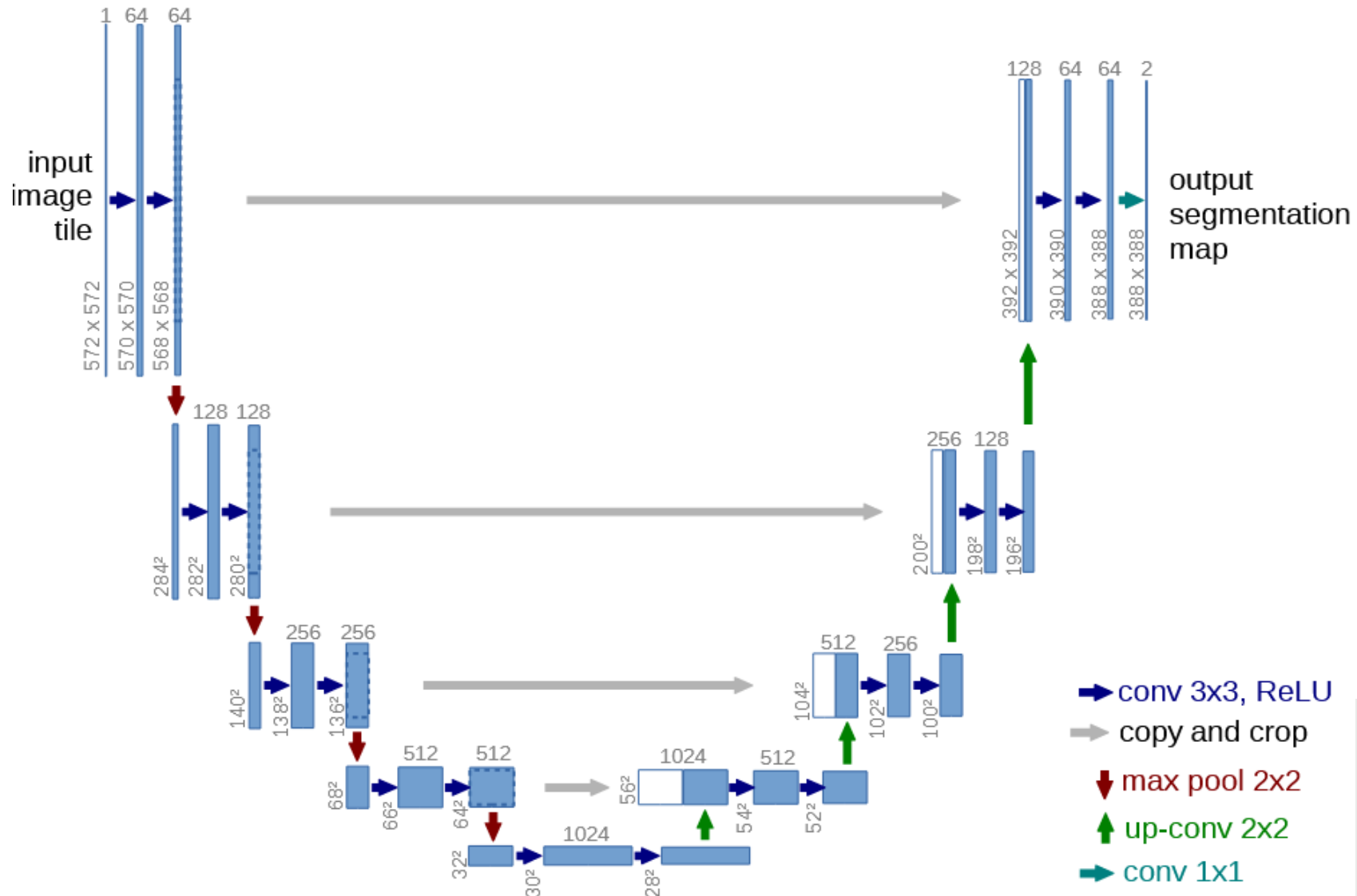
- FCN
- U-Net
- DeepLab
- Mask R-CNN

TITEL	ZITIERT VON	JAHR
U-net: Convolutional networks for biomedical image segmentation O Ronneberger, P Fischer, T Brox International Conference on Medical image computing and computer-assisted ...	139986	2015

**U-Net
extracts
features
through the
encoder,
restores
spatial
detail
through the
decoder,
and
preserves
fine
structure
using skip
connections.**



U-Net is a model of size ..., has ~31 million trainable parameters

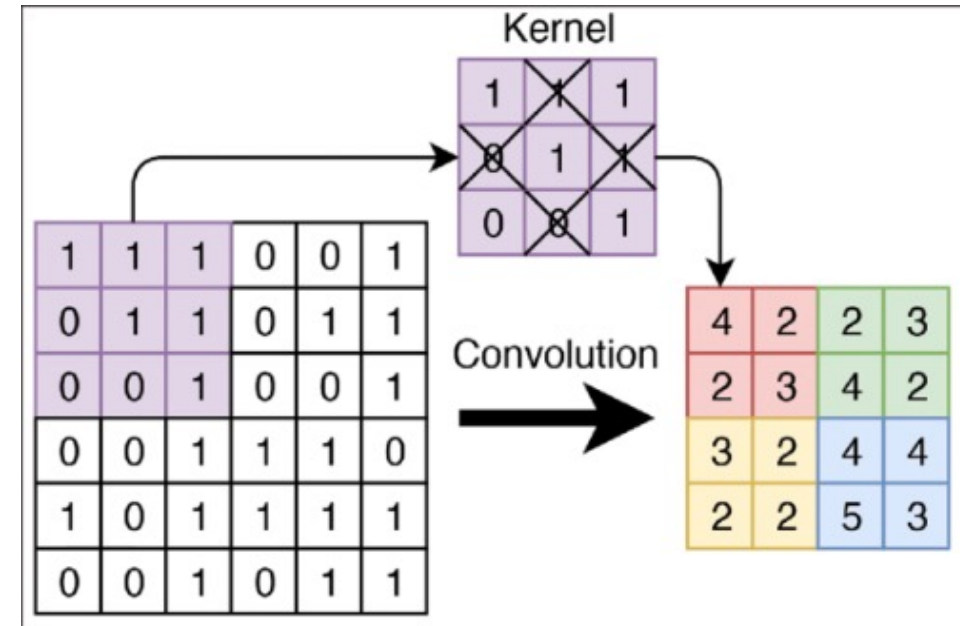


Theoretical Refresher

What is convolution?

Convolution is a local operation where a small kernel (filter) slides over an image (or feature map), computing a weighted sum at each position.

Each neuron in the output (feature map) depends only on a small region (local receptive field) of the input.

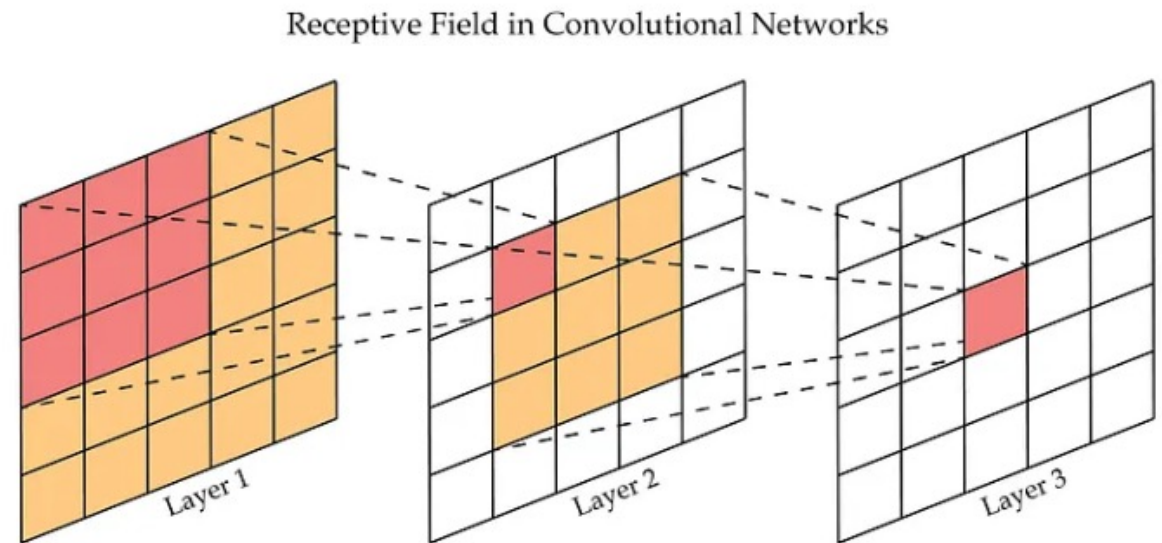


Theoretical Refresher

What is a receptive field?

The receptive field of a convolutional layer in a neural network is the region of the input image that is used to compute the output of a particular neuron in the feature map.

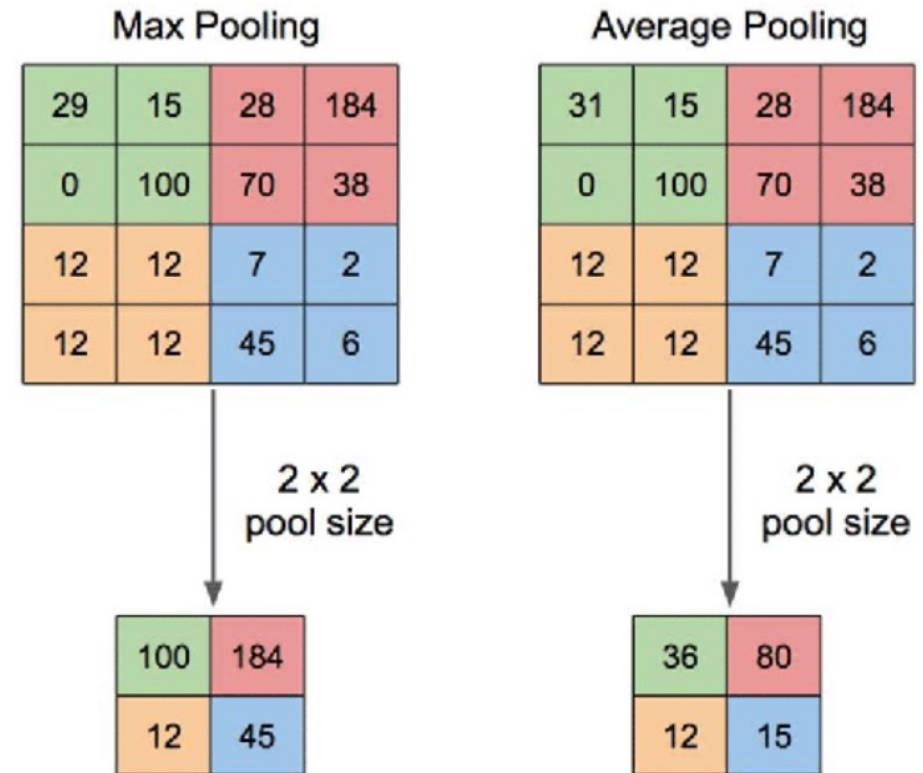
The receptive field is important because it determines the context or the information that the neuron has access to when making a prediction.



Theoretical Refresher

What is pooling?

Pooling (also called sub-sampling) reduces the spatial size of feature maps by summarizing patches (e.g., a 2×2 block) into a single value. Common types: max-pooling (take maximum) and average pooling (take average).



Theoretical Refresher

What is upsampling in general?

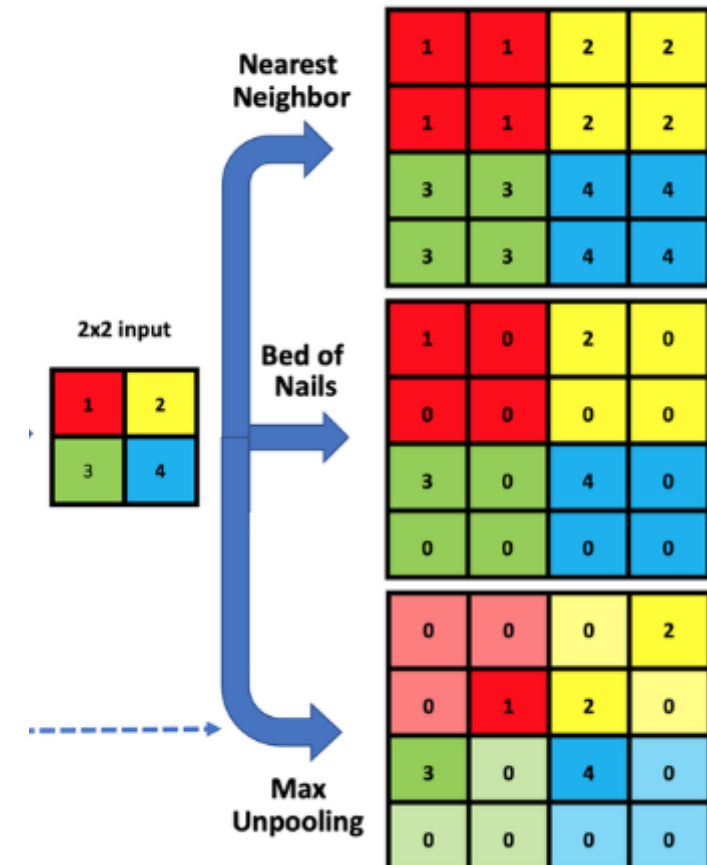
Upsampling increases the spatial resolution of feature maps, contrary to pooling.

Methods include:

- **Nearest Neighbor**
- **Bed-of-Nails**
- **Max Unpooling**
- **Bilinear Interpolation**
- **Bicubic Interpolation**
- **Lanczos Interpolation**

How does upsampling happen in the decoder part?

→ Through the transposed convolution, learnable upsampling



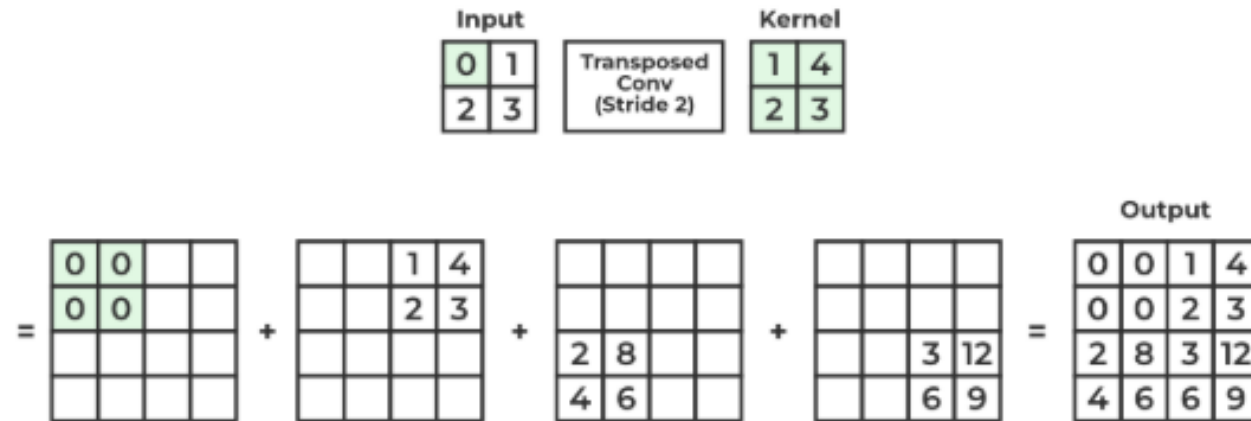
Theoretical Refresher

Transposed Convolution in the decoder:

Transposed convolution is used to upsample feature maps in the decoder by learning how to expand spatial resolution.

How it works internally:

- It upsamples feature maps by inserting zeros between the input elements (based on the stride), creating a “sparser” intermediate map.
- Then a learnable convolutional kernel is applied over this expanded map, producing a higher-resolution output.
- Because the kernel weights are trained, the layer learns how to upsample in a way that’s more expressive than fixed interpolation (“just copying” or bilinear upsampling).
- Note: output size also depends on padding and output_padding, and transposed convs may introduce artifacts if not used carefully.

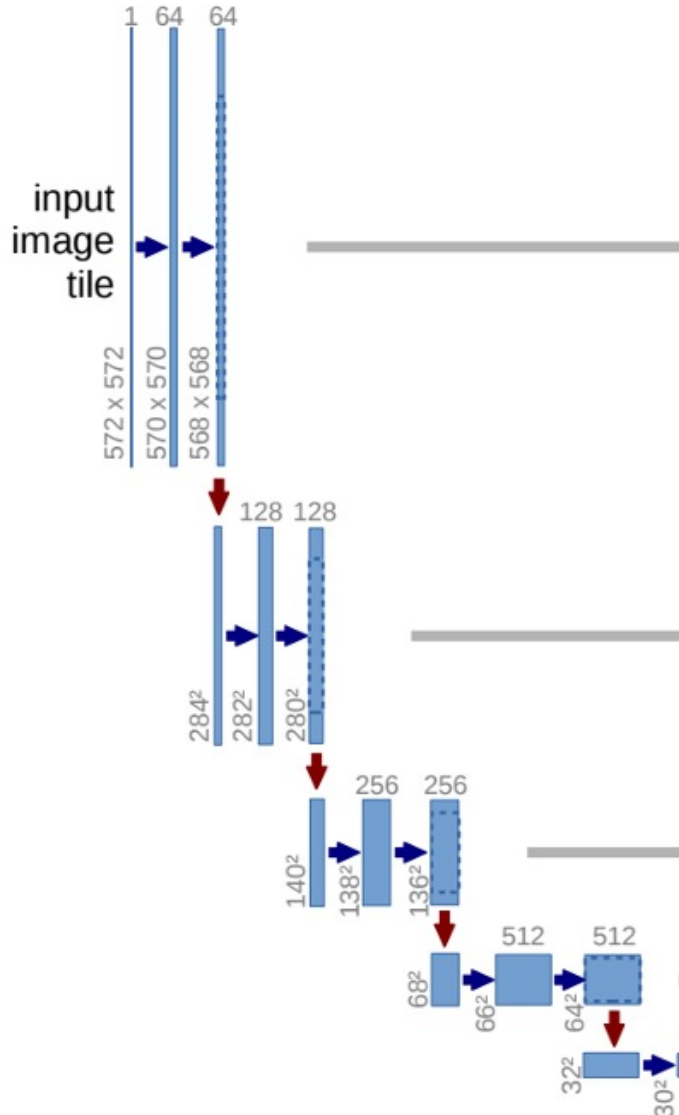


Transposed Convolutional with stride 2

UNet parts

- As we go deeper, feature maps get spatially smaller, so the network compensates by increasing the number of channels to learn richer, more abstract patterns.
- Doubling channels at each stage lets the encoder capture more complex features (edges to textures to shapes to objects) while keeping computation balanced.

Encoder



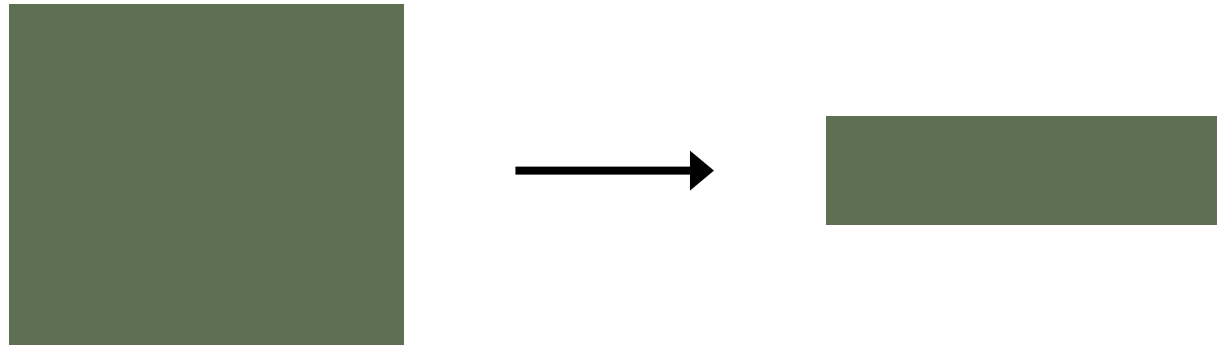
Contracting Path

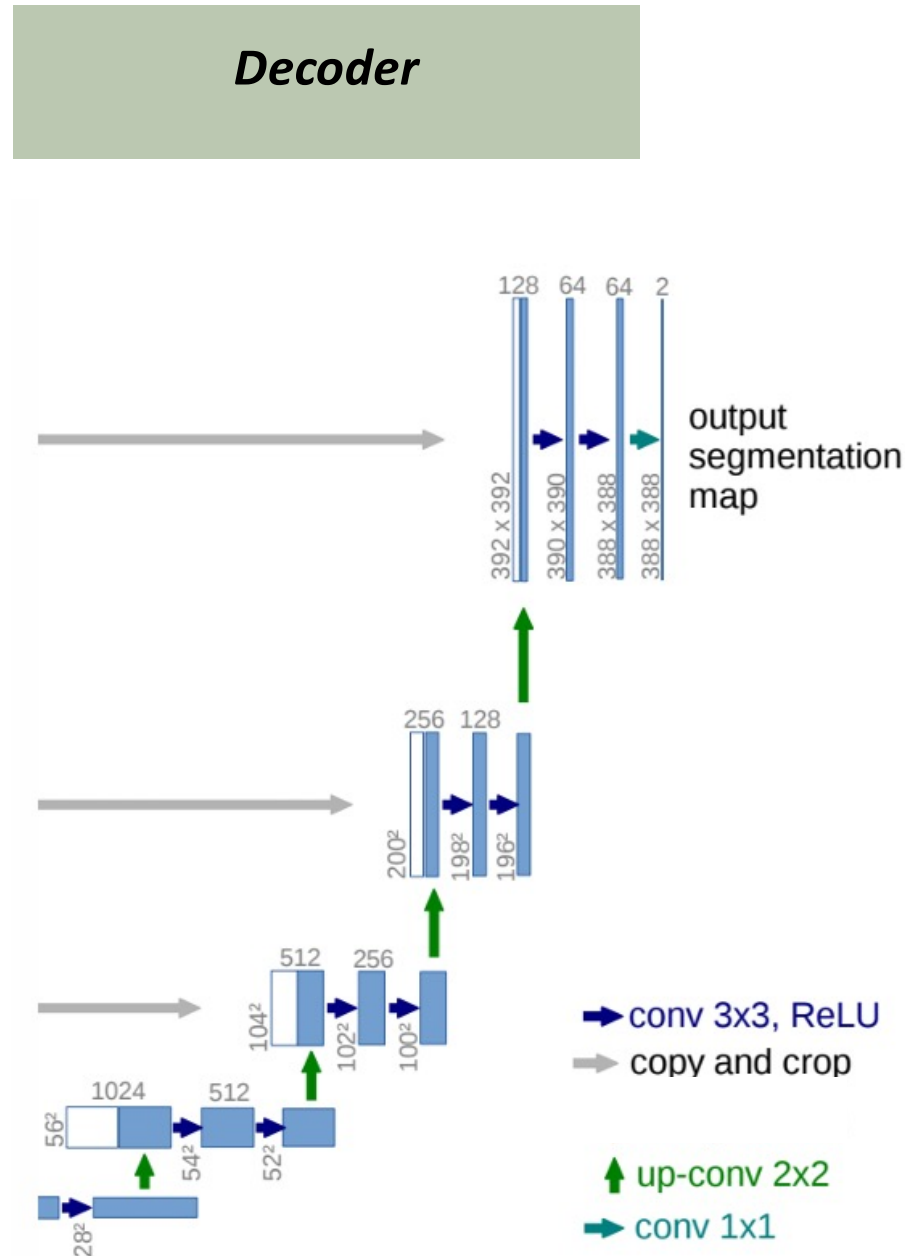
- 3×3 Convolution
- 2×2 Max-Pooling

- conv 3x3, ReLU
- copy and crop
- ↓ max pool 2x2
- ↑ up-conv 2x2
- conv 1x1

UNet parts

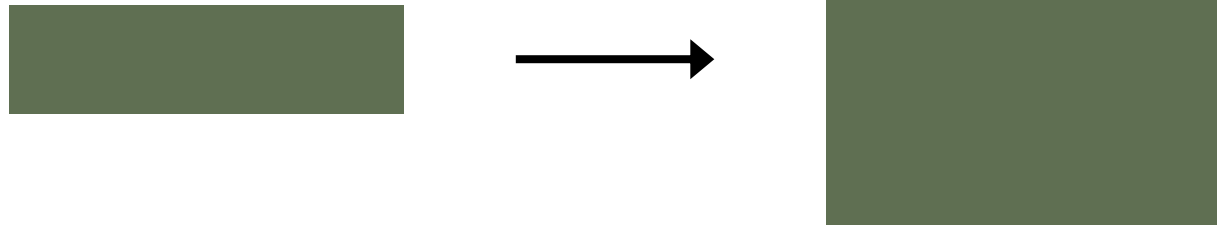
encoder to bottleneck





UNet parts

bottleneck to decoder

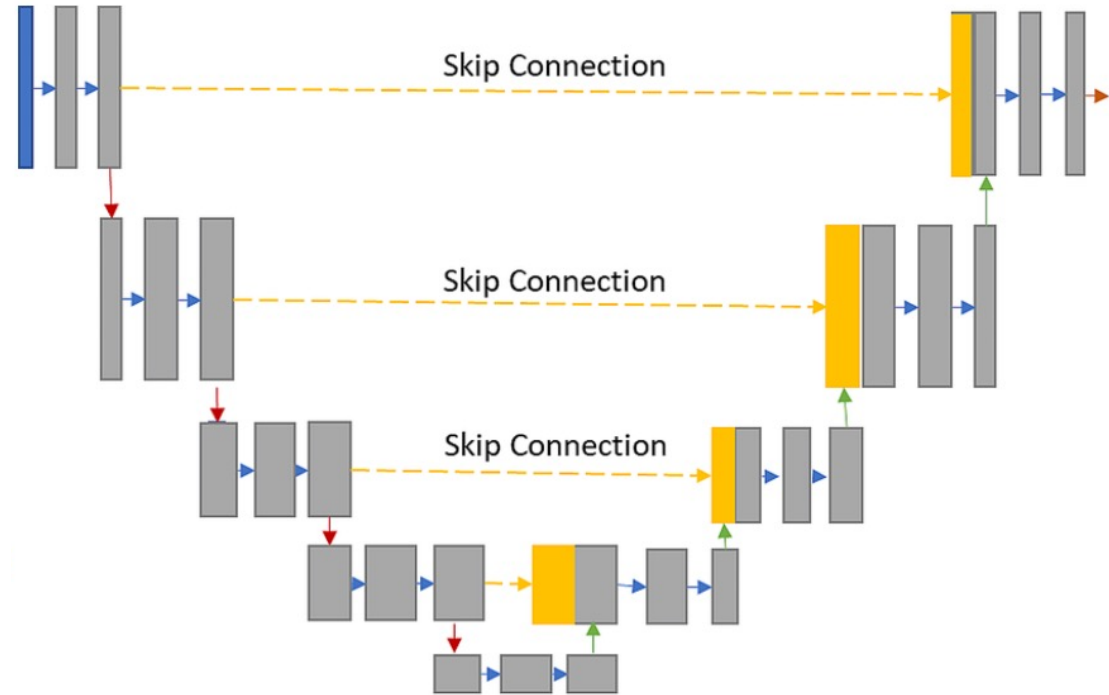


UNet parts

Skip connections transfer high-resolution spatial details from the encoder to the decoder to avoid information loss.

The encoder feature map and the decoder feature map are concatenated along the channel axis.

Skip Connections



- 1*1 Convolution
- Convolution
- Copy
- Pooling
- Upsampling

UNet parts

Skip Connections

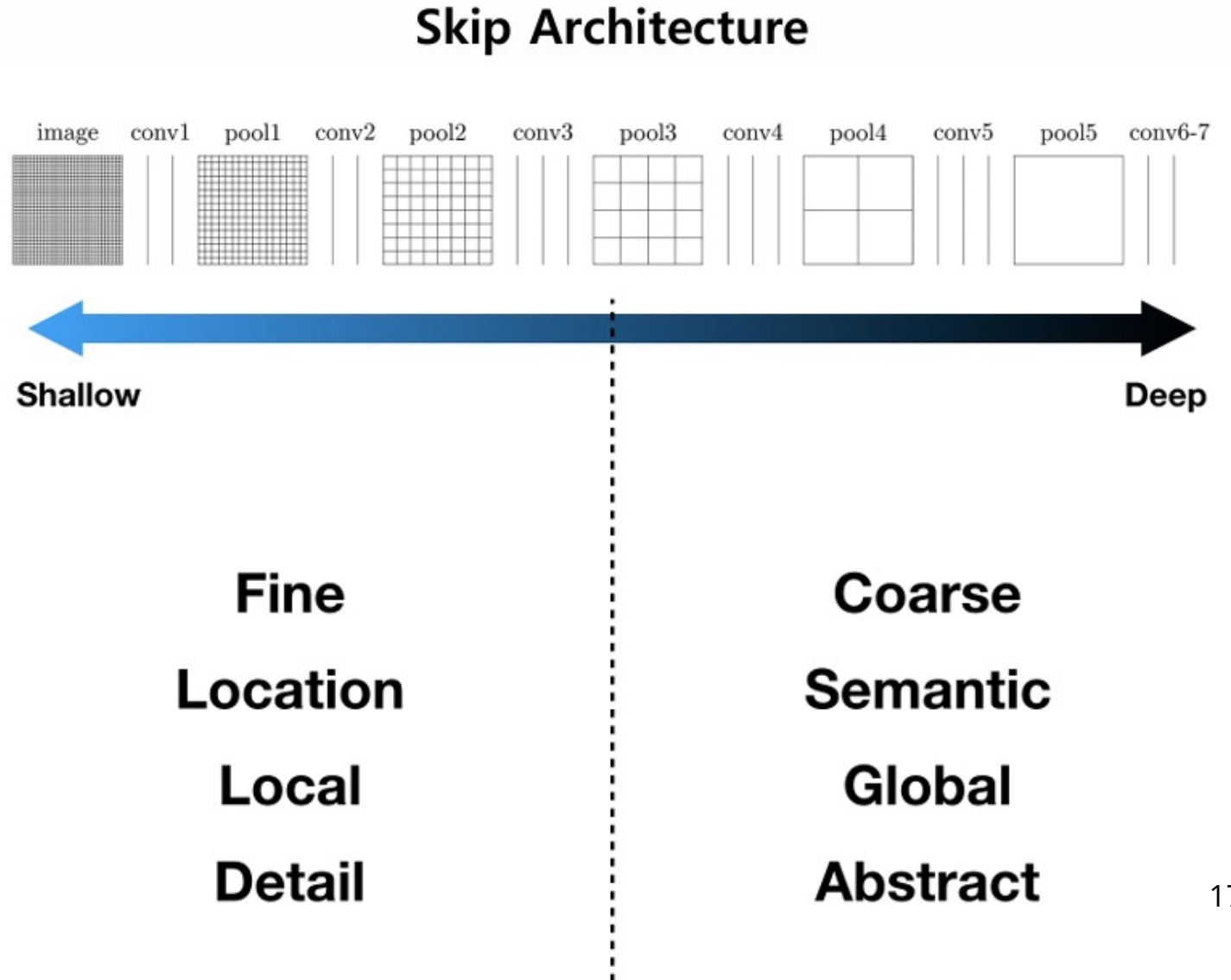


Image Sizes Impact

Example A: Small Input: 256×256

Encoder (Down path)

- After 1st pool: 128×128
- After 2nd pool: 64×64
- After 3rd pool: 32×32
- After 4th pool: 16×16
- At Bottleneck: 16×16

Decoder (Up path)

- After first up: 32×32
- Next: 64×64
- Next: 128×128
- Final: 256×256

Impacts: Less memory, faster computation; but smaller receptive field coverage may limit ability to capture large context unless kernel depth / receptive field is large.

Image Sizes Impact

Example B: Medium Input: 512× 512

Encoder:

- After pool1: 256 × 256
- pool2: 128 × 128
- pool3: 64 × 64
- pool4: 32 × 32
- Bottleneck: 32 × 32

Decoder:

- up1: 64 × 64
- up2: 128 × 128
- up3: 256 × 256
- up4: 512 × 512

Impacts: Standard U-Net config: more memory but also more context captured; many implementations assume multiples of pooling to maintain alignment.

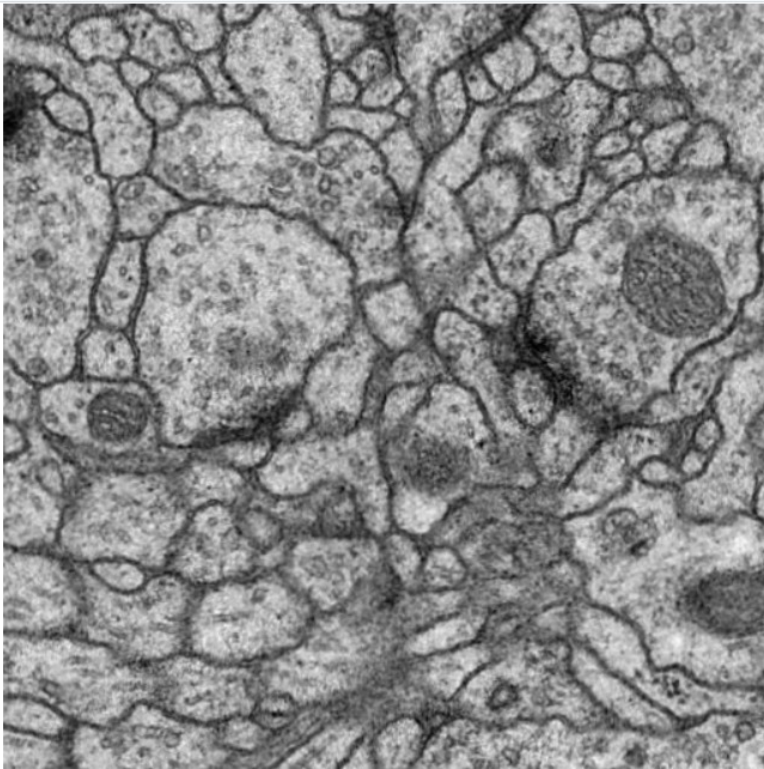
Image Sizes Impact

Conclusion:

- Increasing input size increases spatial dimensions throughout the network (encoder → bottleneck → decoder), leading to higher memory and compute cost.
- Parameter count stays roughly the same regardless of input size, since U-Net uses convolution with fixed filter counts.
- Larger input enables more context (useful for larger structures), but also requires careful design (padding, pooling) and sufficient hardware.

Example

Dataset Example:



train image



train image annotation

Training Settings

Data Augmentation:

U-Net relied heavily on elastic deformation and geometric augmentation to artificially enlarge the tiny medical datasets, becoming a key reason for its strong performance.

What U-Net used:

- Elastic deformations (the most important augmentation)
- Random rotations, translations, scalings.
- Grey-value variations
- Mirroring (flip)

Why:

- Medical datasets were extremely small (only around 30 training images).

Training Settings

U-Net was trained with these parameters :

Optimizer:

Stochastic Gradient Descent (SGD), implemented via the Caffe framework.

Momentum:

- 0.99 to stabilize and speed up training. Very high momentum = 0.99, so updates are strongly influenced by past gradients.

Learning Rate:

- Started high, manually reduced during training (they did not specify the exact value)

Batch Size:

- They use batch size = 1 image per iteration.
- Image Size: 512×512 input tiles (EM images from the ISBI challenge).

Training Settings

Loss Function

During U-Net training, pixel-wise softmax is used to convert logits into class probabilities and optimized the network using cross-entropy loss computed for every pixel in the segmentation map.

$$p_k(x) = \frac{e^{a_k(x)}}{\sum_{k'} e^{a_{k'}(x)}}, \quad H(p, q) = - \sum_x p(x) \log q(x)$$